# Modèles Algorithmiques Fondamentaux sur Structures à Accès Direct

**Thème principal :** Algorithmique et Définition des Parquetages de Boucles en C++

**Synthèse exécutive :** Ce cours pose les bases fondamentales des motifs algorithmiques réutilisables sur les structures à accès direct (tableaux unidimensionnels et chaînes de caractères `std::string` en C++). Il résout un problème récurrent en informatique : comment traiter une collection de données de taille connue ou bornée de façon optimale, lisible et exempte de bugs hors-limite. L'étudiant y apprend à maîtriser la distinction cruciale entre les parcours exhaustifs (traitement de la totalité des éléments via des boucles déterministes) et les parcours sélectifs/interrompus (recherche de première occurrence avec arrêt prématuré dès la condition satisfaite), tout en intégrant les contraintes d'optimisation mémoire et de gestion d'indices.

## 1. Le Parcours Complet avec Traitement Systématique

Pourquoi cette notion existe-t-elle ? Dans de nombreux cas industriels (calcul de statistiques, application d'un filtre d'image pixel par pixel, augmentation générale des salaires dans une base de données), il est indispensable d'appliquer un même traitement à la totalité d'un ensemble de données. En C++, lorsque la taille N de la structure est connue à l'avance, la structure de contrôle la plus adaptée est la boucle 'pour' (`for`). Le parcours commence à l'indice `0` et s'arrête à `taille - 1`. Une erreur fréquente consiste à dépasser la borne supérieure (erreur 'Off-By-One'), conduisant à un comportement indéterminé ou un plantage ('Segmentation Fault').

- Structure à accès direct : Structure de données (ex: `std::vector`, tableau C, `std::string`) où chaque élément est accessible immédiatement en temps constant O(1) via son indice.
- Parcours complet : Traitement obligatoire de chaque élément du premier jusqu'au dernier, sans interruption.
- Invariant de boucle : Propriété logique qui reste vraie à chaque itération du parcours.

***Cas concret enseignant :*** *L'analysant à un contrôle d'appel en classe : l'enseignant doit appeler chaque élève présent sur sa feuille du premier au dernier, sans sauter personne et sans s'arrêter au milieu même s'il trouve un élève spécifique.*

## 2. Le Parcours Complet avec Traitement Conditionné

Dans ce modèle, la totalité du tableau est obligatoirement parcourue, mais une structure conditionnelle `if` est imbriquée dans la boucle `for`. Cela sert typiquement à dénombrer des éléments spécifiques, à faire la somme uniquement des valeurs positives, ou à nettoyer des données invalides. La boucle visite N éléments, mais le bloc interne n'est exécuté que M fois (avec M <= N).

- Traitement conditionné : Visite de tous les éléments du tableau, mais le traitement lourd ou la modification ne s'applique qu'aux éléments respectant un prédicat donné.
- Prédicat : Expression booléenne évaluée pour chaque élément (ex: `element > 0`).

***Cas concret enseignant :*** *La métaphore de la caisse d'oranges : l'agriculteur inspecte toutes les oranges du carton une par une (parcours complet), mais il ne jette dans la poubelle que celles qui présentent des moisissures (traitement conditionné).*

## 3. La Recherche de Première Occurrence (Parcours Interrompu)

Lorsqu'il s'agit d'évaluer la présence d'un élément ou de localiser sa première apparition (ex: vérifier si un mot de passe contient un caractère spécial ou trouver un utilisateur par son ID), parcourir le reste du tableau après avoir trouvé l'élément est un gâchis de ressources processeur. On utilise donc une boucle `while` combinant deux conditions d'arrêt : 1. Ne pas dépasser la taille du tableau (`i < taille`). 2. Ne pas avoir encore trouvé l'élément (`!trouve`). Sortir de la boucle permet ensuite de vérifier la cause de l'arrêt.

• Arrêt prématuré : Interruption immédiate du parcours dès que le résultat cherché est trouvé, afin d'optimiser le temps d'exécution.

• Drapeau booléen (Flag) : Variable d'état servant à mémoriser si la cible a été trouvée.

• Structure `while` : Boucle privilégiée pour les parcours non-déterministes dont la condition d'arrêt dépend d'un événement.

***Cas concret enseignant :*** *Recherche des clés de voiture dans un sac : dès que vos doigts touchent le trousseau de clés, vous vous arrêtez de chercher. Vous ne continuez pas à vider le sac jusqu'au fond.*

## ■ Questions & Interventions des Étudiants en Classe

**Question étudiant :** Pourquoi ne doit-on pas utiliser une boucle `for` avec un `break` pour la recherche de première occurrence ?
**Réponse du professeur :** En programmation structurée et selon les normes académiques du BUT Informatique, l'usage de `break` pour sortir d'une boucle est considéré comme une mauvaise pratique car il crée des sorties multiples indisciplinées. Utiliser une boucle `while` avec une condition explicite (`i < taille && !trouve`) rend le code plus lisible, prouvable mathématiquement et plus facile à débugger.

**Question étudiant :** Quelle est la différence entre `i < taille` et `i <= taille` dans les conditions de boucle C++ ?
**Réponse du professeur :** En C++, les indices de tableau à N éléments vont strictement de `0` à `N - 1`. Si vous écrivez `i <= taille`, la dernière itération tentera d'accéder à `tableau[taille]`, ce qui est une case mémoire hors du tableau. C'est une erreur classique (dépassement de tampon) qui coûte des points au contrôle !

## ■ Exercice & Démonstration d'Application

**Énoncé :** Écrire une fonction C++ `bool contientNegatif(const std::vector& tab)` qui renvoie `true` si le tableau contient au moins un entier strictement négatif, et `false` sinon. La fonction doit utiliser le modèle algorithmique optimal de recherche de première occurrence (pas de parcours inutile).

**Résolution pas à pas :**
1. Initialisation d'un indice `i = 0` et d'une variable booléenne `trouve = false`. 2. Utilisation d'une boucle `while` conditionnée par `(i < tab.size()) && (!trouve)`. 3. Dans la boucle, tester si `tab[i] < 0`. 4. Si oui, passer `trouve` à `true` (la boucle s'arrêtera à l'évaluation suivante). 5. Si non, incrémenter `i` de 1. 6. Retourner la valeur de `trouve` à la fin de la fonction.

**Résultat : Code C++ optimisé : ```cpp #include bool contientNegatif(const std::vector& tab) { std::size_t i = 0; bool trouve = false; while (i < tab.size() && !trouve) { if (tab[i] < 0) { trouve = true; } else { i++; } } return trouve; } ``` Ce code s'arrête dès le premier négatif trouvé et offre une complexité optimale en O(k) où k est la position de la cible.**

## ■ Auto-Évaluation Flash d'Examen

**Q : Quel modèle algorithmique applique-t-on pour calculer la moyenne de toutes les notes d'un étudiant enregistrées dans un tableau ?**

*R : Le parcours complet avec traitement systématique, car il faut obligatoirement additionner la totalité des notes pour avoir la somme totale.*

**Q : Pourquoi la boucle `while` est-elle préférable à la boucle `for` pour chercher si une lettre existe dans un mot ?**

*R : Parce qu'il s'agit d'un parcours interrompu : on doit s'arrêter dès la première occurrence trouvée. La boucle `while` gère naturellement cette condition d'arrêt précoce contrairement à un parcours exhaustif.*

**Q : Si un tableau `T` contient 5 éléments, quels sont les indices valides pour y accéder en C++ ?**

*R : Les indices valides sont 0, 1, 2, 3 et 4.*